البحث الثنائي
(Binary Search)

كيف نبحث بذكاء في نصف الوقت؟

مرتكزٌ على فكرة واحدة: **التخلّص من نصف المصفوفة في كل خطوة**

إعداد: الأستاذ محمود اغبارية

📖 فكرة من الحياة اليومية

تخيّل أنك تبحث عن كلمة في قاموس يحتوي على **1000 صفحة**. هل تبدأ من الصفحة الأولى وتقرأ صفحة صفحة؟

بالطبع لا! أنت تفتح القاموس من **المنتصف**، وتقرر:

- إذا الكلمة التي تبحث عنها تأتي بعد الصفحة المفتوحة ← تتجاهل **النصف الأيسر** كله.
- إذا تأتي قبلها ← تتجاهل **النصف الأيمن** كله.
- تكرر هذا حتى تجد الكلمة.

💡 هذا بالضبط هو البحث الثنائي!

في كل خطوة تحذف **نصف** العناصر المتبقية. فبدلاً من فحص 1000 عنصر واحداً واحداً، تحتاج فقط إلى **10 خطوات** (لأن $2^{10} = 1024$).

⚠️ شرط أساسي: هذه الفكرة تعمل فقط لأن القاموس **مرتّب أبجدياً**. بدون ترتيب، لا يمكن أن نقرر أي نصف نتجاهله!

خطوات الخوارزمية

البحث الثنائي يعمل على مصفوفة **مرتبة تصاعدياً** كالتالي:

1. نُعرّف ثلاثة متغيرات: `lo = 0`، `hi = arr.Length - 1`، `mid`.
2. طالما `lo ⩽ hi`، نحسب الوسط: `mid = (lo + hi) / 2`
3. نقارن `arr[mid]` مع القيمة المطلوبة `target`:
   - إذا `arr[mid] == target` → **وجدنا! نُعيد mid.**
   - إذا `arr[mid] < target` → نرفع الحد الأدنى: `lo = mid + 1`
   - إذا `arr[mid] > target` → نخفض الحد الأعلى: `hi = mid - 1`
4. إذا خرجنا من الحلقة بدون إيجاد → **نُعيد -1.**

اللون الأخضر = lo | **اللون الأحمر =** hi | **اللون الأصفر =** mid | **رمادي باهت =** نطاق محذوف

محاكاة: ابحث عن target = 33 في المصفوفة

نبدأ: lo=0، hi=9. المصفوفة جاهزة.

target = 33 | mid = - | hi = 9 | lo = 0

0	1	2	3	4	5	6	7	8	9
4	8	12	17	22	28	33	41	50	60
▲ lo									▲ hi

كود C# الكامل

```
public static int BinarySearch(int[] arr, int target)
{
    int lo  = 0;
    int hi  = arr.Length - 1;

    while (lo ≤ hi)
    {
        int mid = (lo + hi) / 2;

        if (arr[mid] == target)
            return mid;          // وجدنا → نُعيد الفهرس

        else if (arr[mid] < target)
            lo = mid + 1;        // المطلوب في النصف الأيمن

        else
            hi = mid - 1;        // المطلوب في النصف الأيسر
    }

    return -1;                   // لم نجده
}
```

ماذا تُعيد العملية؟ إذا وُجد العنصر تُعيد **فهرسه** (index). إذا لم يُوجد تُعيد **-1** — وهي الاتفاقية المعيارية للدلالة على "غير موجود".

⚖️ مقارنة: البحث الثنائي مقابل البحث الخطي

البحث الخطي عن 33: نبدأ من الفهرس 0.

4	8	12	17	22	28	33	41	50	60
0	1	2	3	4	5	6	7	8	9

المعيار	بحث خطي	بحث ثنائي
شرط الاستخدام	**أي مصفوفة**	مرتبة فقط
أسوأ حالة (100 عنصر)	100 مقارنة	**7 مقارنات**
أسوأ حالة (1,000,000)	مليون مقارنة	**20 مقارنة**
السرعة	بطيء على المصفوفات الكبيرة	**سريع جداً**

تمرين 1: متابعة

معطاة المصفوفة المرتبة التالية:

2	5	8	12	16	23	38
0	1	2	3	4	5	6

الفهارس:

نبحث عن `target = 23`. كم مقارنة يحتاجها البحث الثنائي حتى يجد العنصر؟ وما هو الفهرس الذي سيُعاد؟

تتبع الخطوات يدوياً، ثم انتقل للشريحة التالية.

إجابة تمرين 1

3 مقارنات فقط، يُعاد الفهرس 5

التتبع:

- **الخطوة 1:** lo = 4 → lo=0، hi=6 → mid=3 → arr[3]=12 < 23
- **الخطوة 2:** lo=4، hi=6 → mid=5 → arr[5]=23 == 23 → **وجدناه! نُعيد 5 ✓**

> في الواقع احتجنا مقارنتين فقط للوصول إليه. البحث الخطي كان سيحتاج **6 مقارنات** للوصول للعنصر نفسه.

? تمرين 2: حالة "غير موجود"

نفس المصفوفة:

2	5	8	12	16	23	38

سؤال: ماذا ستُعيد العملية إذا بحثنا عن `target = 10`؟ وكيف نعرف أن العنصر غير موجود؟

تتبع الخطوات وراقب متى يصبح `lo` أكبر من `hi`.

إجابة تمرين 2

العملية تُعيد -1

التتبع:

- **الخطوة 1:** arr[3]=12 > 10 → mid=3 → lo=0، hi=6 → `hi = 2`
- **الخطوة 2:** arr[1]=5 < 10 → mid=1 → lo=0، hi=2 → `lo = 2`
- **الخطوة 3:** arr[2]=8 < 10 → mid=2 → lo=2، hi=2 → `lo = 3`
- **الآن:** lo=3 > hi=2 → نخرج من الحلقة → **نُعيد -1**

شرط `lo ⩽ hi` هو "صمام الأمان" الذي يوقف الحلقة عندما يضيق النطاق ونُدرك أن العنصر غير موجود.

🏆 تمرين 3: تحدي البرمجة

الكود الأساسي يبحث في مصفوفة مرتبة **تصاعدياً**.

ما هو التعديل المطلوب في الكود لكي يعمل مع مصفوفة مرتبة تنازلياً؟

```
else if (arr[mid] ??? target)
    lo = mid + 1;
else
    hi = mid - 1;
```

تلميح: في الترتيب التنازلي، العناصر الأكبر تكون في اليسار. كيف يؤثر ذلك على قرارنا؟

إجابة تمرين 3

نعكس الشرط: نغير < إلى >

```
else if (arr[mid] > target)
    lo = mid + 1;   // المطلوب في النصف الأيمن (الأصغر)
else
    hi = mid - 1;   // المطلوب في النصف الأيسر (الأكبر)
```

التفسير:

في الترتيب التصاعدي: إذا arr[mid] أصغر من target → نتجه يميناً (نرفع lo).

في الترتيب التنازلي: إذا arr[mid] أكبر من target → نتجه يميناً أيضاً (لأن اليمين أصغر الآن).

نهاية الدرس

كل التوفيق والنجاح!

✓ **البحث الثنائي:** في كل خطوة يُقلّص مساحة البحث إلى النصف، مما يجعله سريعاً جداً حتى على مصفوفات ضخمة.

⚠ **الشرط:** المصفوفة **يجب** أن تكون مرتبة.

أرجو لكم عملاً ممتعاً! / الأستاذ محمود اغبارية